When Software Fabricates Assurance: Seven Design Principles for Trustworthy Governance, Risk and Compliance Systems

Name of the Track: Track 1 – Information Systems: Theory and Practice (Digital Ethics and Policy: Regulation and Compliance)

Type of Submission: Ongoing research (design science research; artifact and technical evaluation completed, naturalistic evaluation pending)

Abstract

1. Purpose

Boards, auditors and regulators rely on governance, risk and compliance (GRC) information systems for evidence that risks are identified, rated, approved and controlled. Research on these systems has examined their adoption and value. We found no study asking whether the evidence they present can itself be trusted. A code and runtime audit of an existing risk management application found that the software invented risk ratings, discarded approvals, accepted unauthenticated authority and reported failed database writes as successes. This study asks which design failures compromise the integrity of assurance evidence. It also asks which design principles prevent them, and how well an instantiation of those principles performs against the original system.

2. Design/Methodology/Approach

This is a design science study. We read the code of an existing risk management application, ran it, and recorded seven distinct ways in which it failed. Each failure was traced to theory that already explains it, from integrity models, fail-safe defaults and complete mediation to separation of duties, tamper-evident logging and the critique of audit as ritual, and a design principle was drawn from that theory. We built a new system, ERMA-GRC, on these principles. The old and new systems then faced 23 scenarios over the same stored data, under a protocol time-stamped in the project repository beforehand. Eighteen scenarios come from the audit, four go beyond it, and one was meant to fail. All deviations are reported.

3. Findings

Of the 18 scenarios taken from the audit, the original system stopped the failure in 4. ERMA-GRC stopped it in all 18. It refused writes without a session, would not let anyone approve their own work, stamped each audit entry with the signed-in user instead of a claimed name, caught an edited entry, and showed a database outage as an error. Beyond the audit, it stopped 3 of 4 failures, including two approvers deciding on one record at the same moment. Stored outcomes were read from the database stand-in, not from what a system claimed. The new design failed twice, and we report both in full. A key risk indicator showed the right status on screen, yet a client calling the server directly could store any status it chose. And anyone holding the audit key could rewrite the trail offline, unnoticed after a restart. The principles also lean on each other: separation of duties is only as sound as the audit trail it reads, and a rule checked only in the browser assures nothing.

This Abstract is submitted to LEAD-2027 conference conducted at IIM Shillong during 20-22 January 2027

4. Originality/Value

We treat the integrity of assurance evidence as something a GRC system's design can protect or destroy, rather than as an organisational outcome. The paper offers a taxonomy of failures seen in a working system, seven principles with two testable propositions towards a design theory, and a scenario protocol that others can rerun. Auditors and buyers can use the principles as a checklist. The two failures set the next tasks: anchoring audit trails outside the system, computing every metric on the server, and testing the principles with practitioners.

Keywords

assurance integrity; GRC information systems; design science; enterprise risk management; tamper-evident audit trail; separation of duties